

ASSESSMENT TOOL 3
WEB-DEVELOPMENT
C.PAVANI
CSA1006-SOFTWARE ENGINEERING

Problem 1: Full-Stack Web App Development for an E-Commerce Dashboard

Scenario

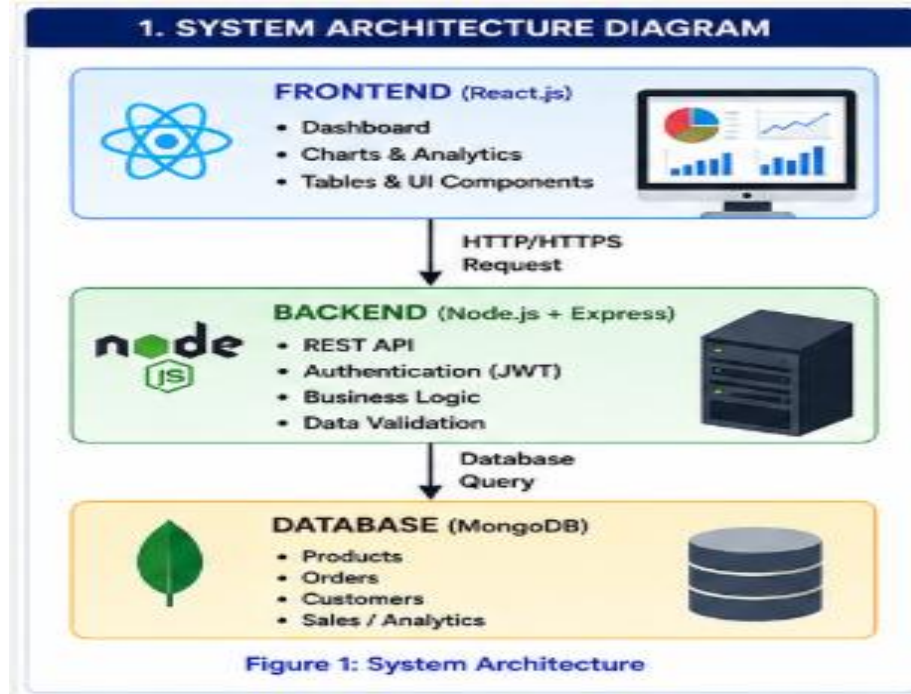
A software development company is building a **Full-Stack E-Commerce Analytics Dashboard** to help marketing teams and management monitor business performance in real time. The frontend is developed using **React.js**, while the backend uses **Node.js with Express.js**. Data is stored in **MongoDB** and retrieved through REST APIs. The application must display interactive charts, recent orders, customer statistics, and product analytics with a clean and responsive user interface.

Objective

The objective of this project is to develop a responsive web dashboard that:

- Displays real-time sales analytics.
- Fetches data through REST APIs.
- Provides interactive charts and tables.
- Supports collaborative development using Git.
- Uses Continuous Integration (CI) for automated testing and deployment.

System Architecture



Components Explanation

1. React Frontend

The React application provides the graphical user interface for users.

Features

- Dashboard
- Sales Analytics
- Product List
- Customer Details
- Order Management
- Responsive UI
- Interactive Charts

Technologies Used

- React.js
- HTML5
- CSS3
- JavaScript
- Axios
- Chart.js

2. Backend API (Node.js + Express)

The backend processes client requests and communicates with the database.

Responsibilities

- User Authentication
- Authorization
- API Routing
- Business Logic
- Data Validation
- Database Operations

Technologies

- Node.js
- Express.js
- JWT Authentication
- REST API

3. Database (MongoDB)

MongoDB stores all application data.

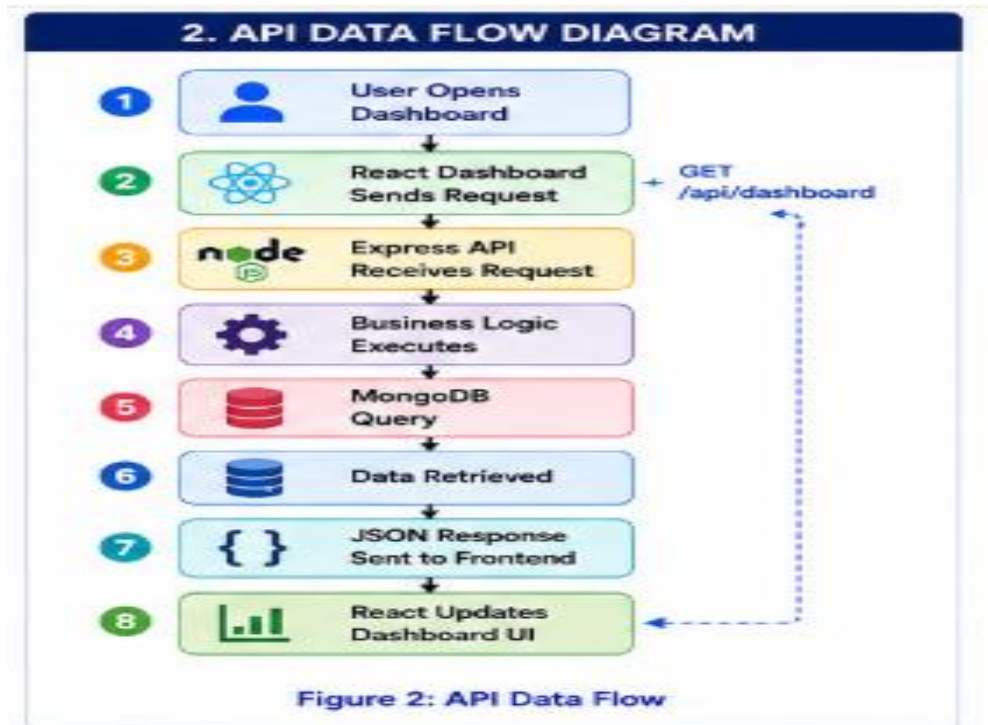
Collections include

- Users
- Products
- Orders
- Customers
- Sales Reports

Advantages

- NoSQL Database
- High Performance
- Flexible Schema
- Easy Scalability

API Data Flow



Explanation of API Data Flow

Step 1

The user opens the React dashboard.

Step 2

React sends an HTTP GET request to the backend API.

Example:

GET /api/dashboard

Step 3

Node.js receives the request.

Step 4

Express validates the request.

Step 5

MongoDB retrieves sales information.

Step 6

The backend converts the data into JSON format.

Step 7

The JSON response is sent to React.

Step 8

React updates the dashboard charts and tables automatically.

Git Branching Strategy (Feature Branching)



Branch Description

Main Branch

- Production-ready code
- Stable version
- Protected branch

Develop Branch

- Integration branch
- Tested features are merged here

Feature Branch

Each developer creates a separate feature branch.

Examples

feature/login

feature/orders

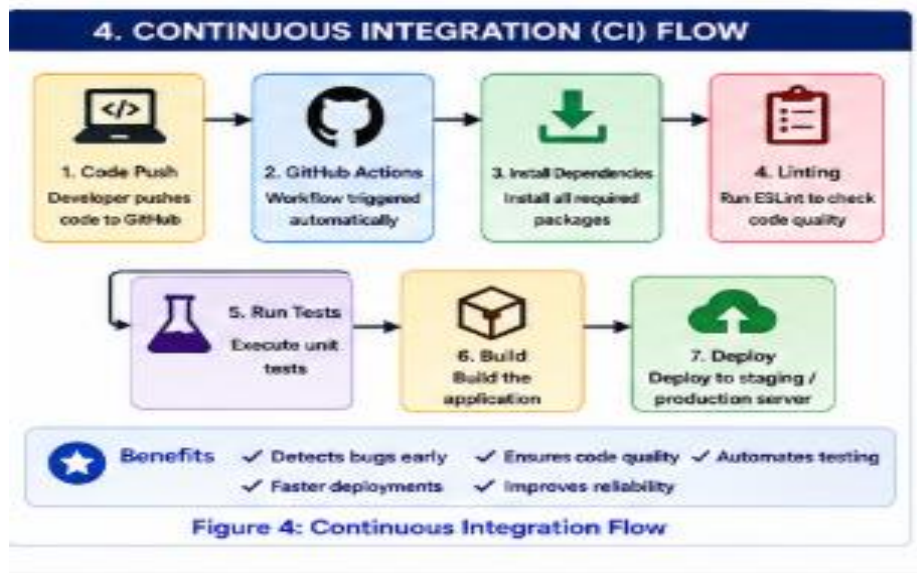
feature/dashboard

feature/charts

Advantages

- Easy collaboration
- Fewer conflicts
- Better code management

Continuous Integration (CI) Flow



CI Workflow Explanation

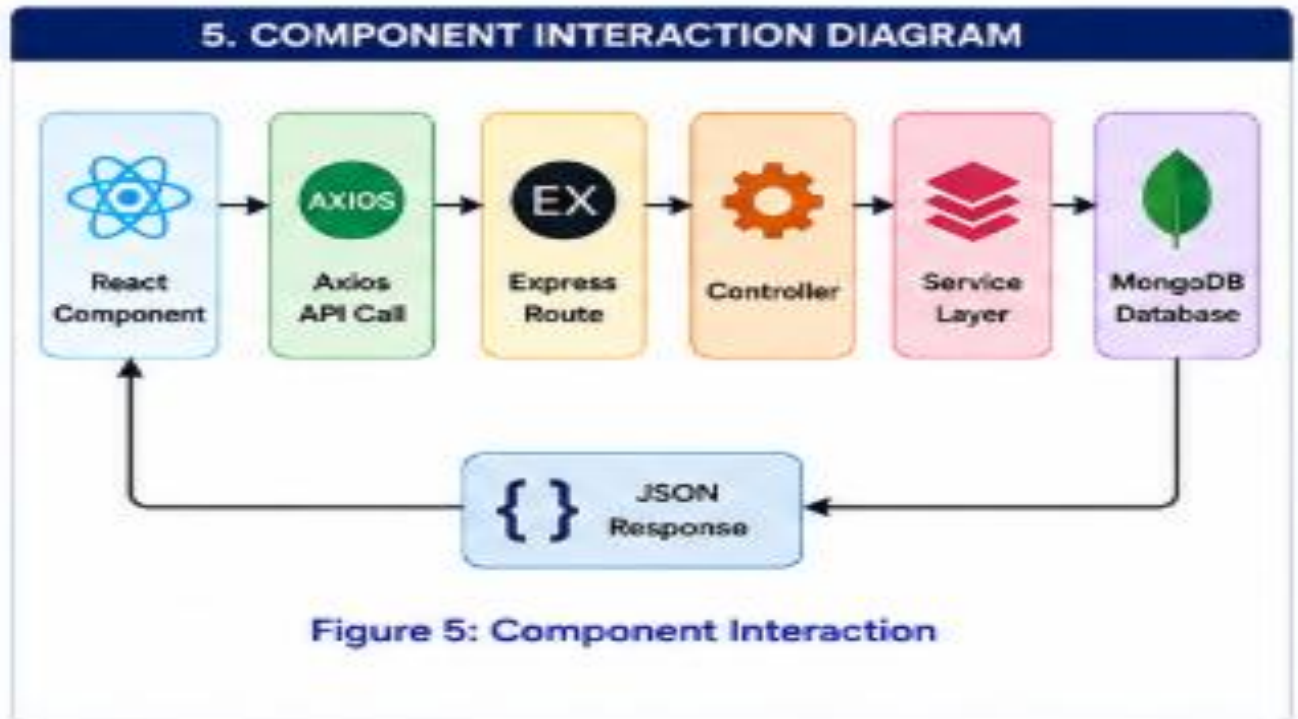
The CI pipeline performs the following tasks:

- Downloads project dependencies
- Executes ESLint
- Runs automated unit tests
- Builds the React application
- Deploys the application if all tests pass

Benefits

- Detects bugs early
- Improves code quality
- Saves development time
- Ensures stable deployments

Component Interaction



Example Pull Request Review Checklist

Checklist Item	Status
----------------	--------

Code compiles successfully	✓
----------------------------	---

No syntax errors	✓
------------------	---

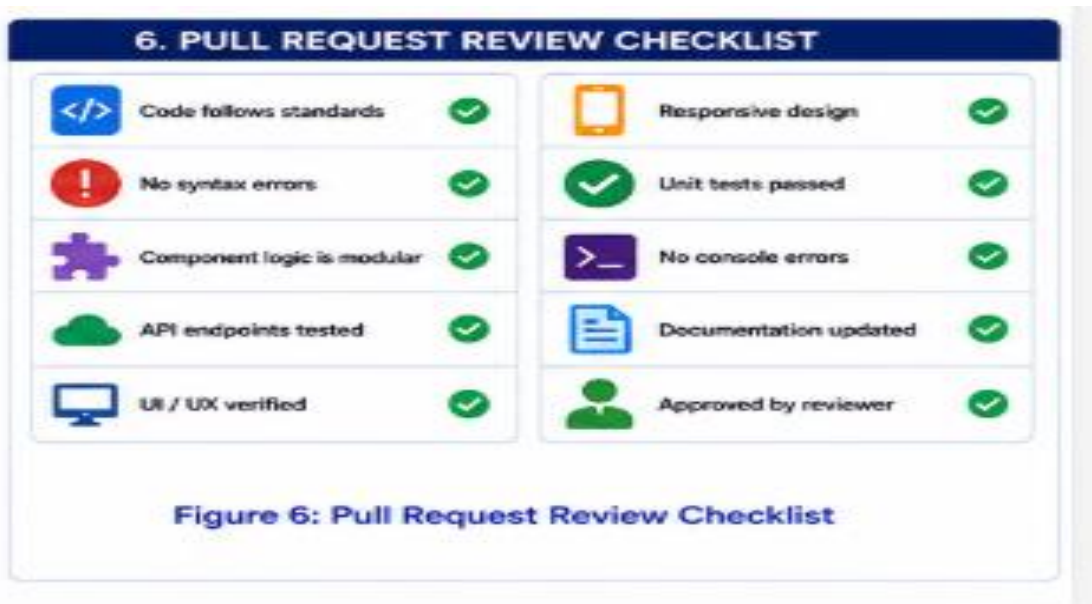
Coding standards followed	✓
---------------------------	---

API endpoints tested	✓
----------------------	---

UI works correctly	✓
--------------------	---

Responsive design verified	✓
----------------------------	---

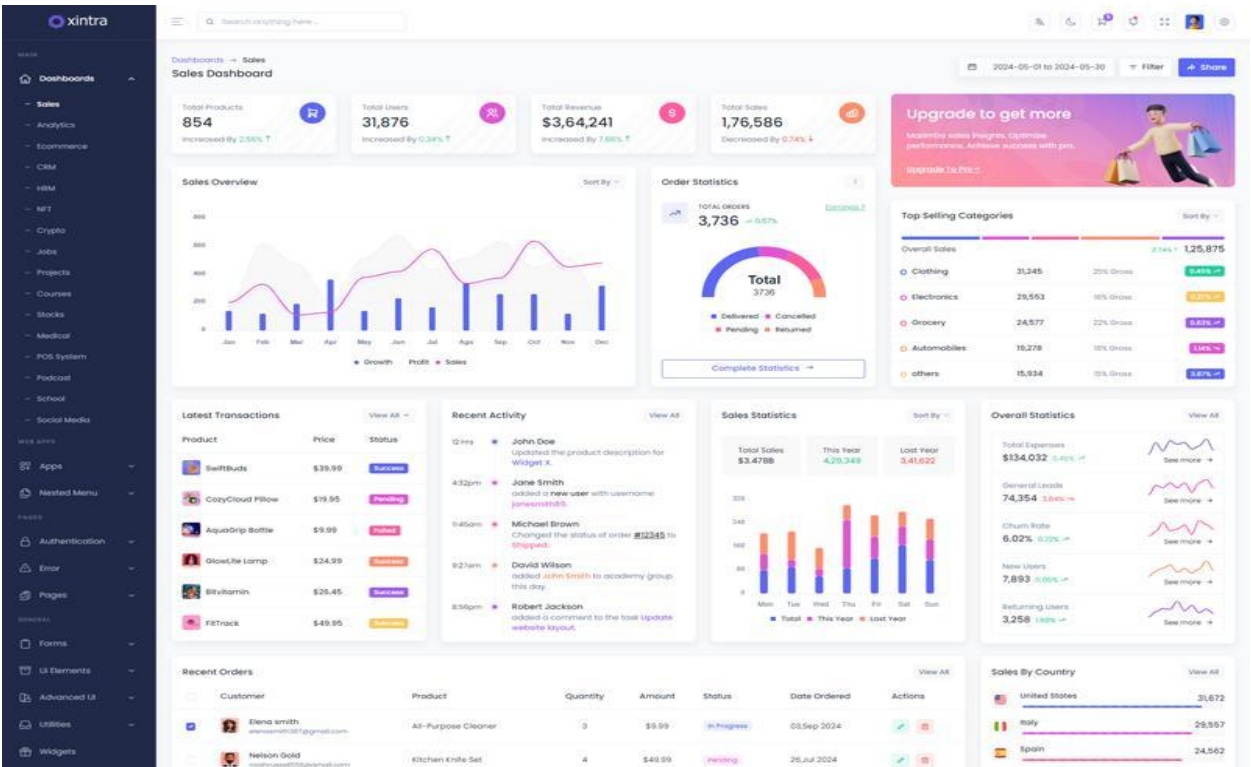
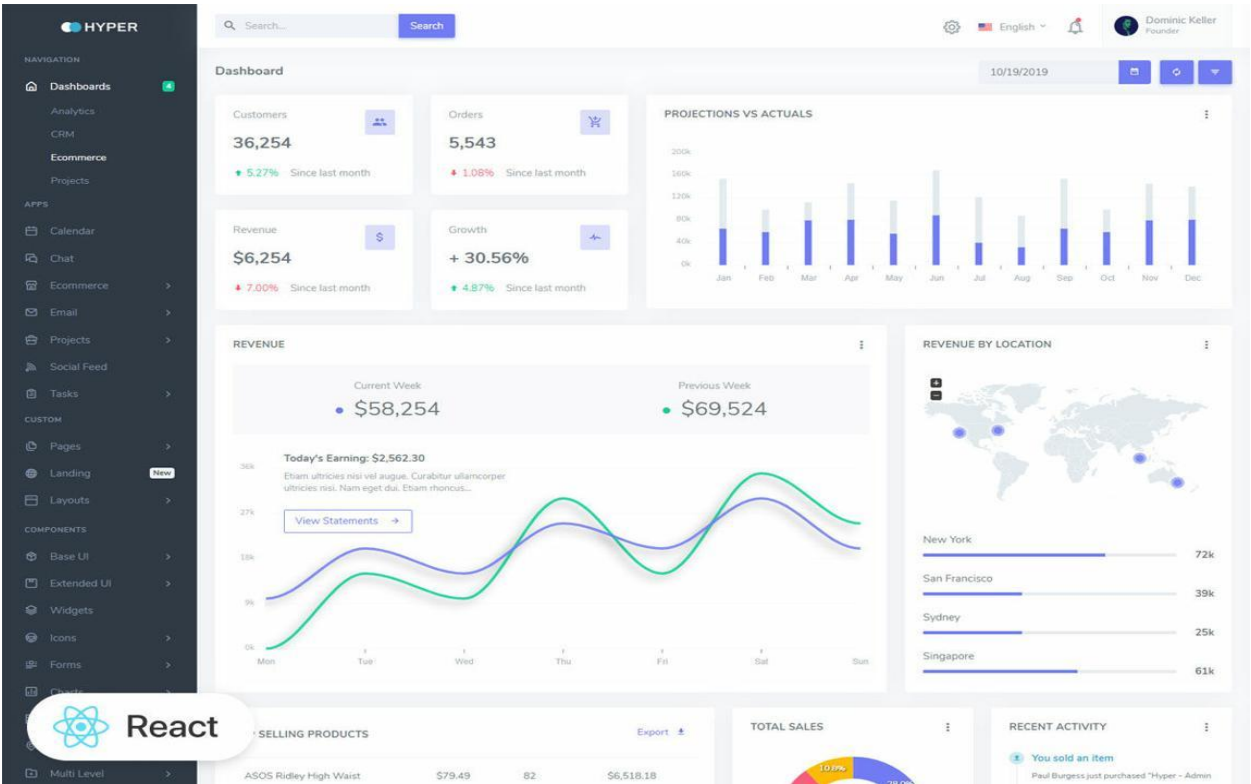
Checklist Item	Status
Unit tests passed	✓
No console errors	✓
Documentation updated	✓
Approved by reviewer	✓



Advantages of the Proposed Solution

- Fast user interface using React
- Real-time API communication
- Secure backend using Express.js
- Scalable MongoDB database
- Easy team collaboration through Git
- Automated testing with CI/CD
- Responsive and modern dashboard

Sample Web Dashboard Screenshots (Insert in Word Document)





Conclusion

The proposed Full-Stack E-Commerce Dashboard combines **React.js**, **Node.js**, **Express.js**, and **MongoDB** to create a modern, scalable, and responsive web application. By implementing Feature Branching, Continuous Integration, and REST APIs, the system supports efficient team collaboration, faster development, and reliable deployment. The architecture ensures real-time analytics, improved maintainability, and an enhanced user experience, making it suitable for enterprise-level e-commerce applications.